

Exercise 1 - Quantum Computing Simulator

Some Preliminary Remarks

- **The exercise is voluntary, the solutions do not have to be handed in.** The solution will be discussed on 08.06.2026 after the lecture.
- The goal of the exercise is to deepen the theoretical contents of the lecture. We will use different tools to implement and simulate gate based quantum algorithms (and if you work the sheet to the end we will even see how to use a real device). These tools might be useful for your exam preparation and also for your possible future work with quantum computers.
- The exercise can be worked on with the knowledge from the lecture and some basic Python know-how. The tools we use in this exercise are very common, there is a lot of documentation and help available online if you are stuck at some point.
- Feel free to implement other algorithms from the lecture or your own ideas, it is the best way to learn about quantum computing.

1 Simulation with Basic Linear Algebra

To simulate an ideal quantum computer, we don't need a complicated simulator. The states are described by vectors in a complex vector space and the operations are multiplications with complex matrices. Therefore we can do the simulation by hand ("pen and paper" simulation). But since there is more to life than doing matrix multiplications by hand, we will let a computer program do the job for us which is capable of dealing with complex vectors and matrices.

One such program is *SageMath* which we will use for the first part of this exercise. SageMath is a collection of different libraries for mathematics like *NumPy*, *Sympy* and *matplotlib*. The syntax is almost identical to Python. If you are familiar with Python, using SageMath should be straightforward.

The software can be downloaded from the [SageMath website](#) for Linux, Windows and macOS. For lots of Linux distributions it is also available via the package manager. See the [SageMath installation guide](#) for detailed instructions.

For the tutorial sessions the recommended way of programming is to use a virtual environment, e.g. [Anaconda](#). The environment yaml is available in the moodle course, creating the environment is therefore just a `conda env create -f environment.yml`.

SageMath can be started as kernel for a Jupyter Notebook, which is the recommended way of using it. The solutions to this exercise will be provided in a Jupyter Notebook as well. Once you have successfully created the environment, we are ready for the first task.

1.1 Base States

A state in quantum computing is represented by a ket, which is a column vector. As you learned in the lecture, the two 1-qubit base states of the z-base are:

$$S_z^+ : |0\rangle \hat{=} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ and} \quad (1)$$

$$S_z^- : |1\rangle \hat{=} \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (2)$$

We can write them down in SageMath as below. The function `pretty_print` can be used to display formulas and matrices nicely.

```
1 z_0 = matrix([1, 0]).transpose()
2 z_1 = matrix([0, 1]).transpose()
3 pretty_print(z_0, z_1)
```

A two qubit base state can be expressed as the tensor product of two one qubit base states. We can for example calculate the state $|00\rangle$ as tensor product of $|0\rangle$ and $|0\rangle$:

```
1 z_00 = z_0.tensor_product(z_0)
2 pretty_print(z_00)
```

Exercise 1:

- (a) Calculate the other three base states $|01\rangle$, $|10\rangle$ and $|11\rangle$. Does the result match what you learned from the lecture?
- (b) Write a function which outputs all base states in the z base for n qubits. You can use normal Python syntax to declare functions, loops etc.

The base states of x- and y-base can also be expressed in terms of the z-base:

$$S_x^+ : |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \hat{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (3)$$

$$S_x^- : |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \hat{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad (4)$$

$$S_y^+ : |+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \hat{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \quad (5)$$

$$S_y^- : |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) \hat{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix} \quad (6)$$

We can express for example the state $|+i\rangle$ in SageMath like shown below. Note that the imaginary unit i is expressed with a capital I in SageMath, since the lower case i is often used for indices.

```
1 y_0 = (1 / sqrt(2)) * matrix([1, I]).transpose()
2 pretty_print(y_0)
```

Exercise 2:

- (a) Write down the other states $|+\rangle$, $|-\rangle$ and $| -i\rangle$ in SageMath.
- (b) Extend the function from Exercise 1 to be able to produce multiple qubit states in an arbitrary base. The chosen base can be a second parameter of the function.

1.2 Quantum Gates

We can express quantum gates as matrices. Below is an example for the Hadamard gate. To calculate the result, we multiply a state vector with the matrix. Note that gates are applied from the left.

```
1 H = (1 / sqrt(2)) * matrix([[1, 1], [1, -1]])
2 pretty_print(H, H * z_0, H * z_1)
```

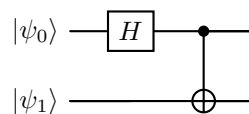
Exercise 3:

- (a) What do we get after applying the Hadamard gate to the states $|0\rangle$ and $|1\rangle$?
- (b) What happens if we apply the Hadamard gate again? You can either test this or already know the answer from the lecture.
- (c) Implement the Pauli-X, Y and Z gate. Apply the Pauli-X gate to the states $|0\rangle$ and $|1\rangle$. Why is this gate also called “not gate”?

Two and more qubit gates can either be defined with larger matrices, or by taking the tensor product of two smaller gates. Reminder: To get the equivalent for two parallel gates, we have to take the tensor product of the two gates. If there is only one gate and a parallel wire, we take the tensor product with the 2×2 identity matrix.

Exercise 4:

We consider the following 2-qubit quantum circuit:



- (a) Calculate the matrix which represents the whole circuit. Start by defining the controlled not gate as a 4×4 matrix and the Hadamard gate parallel to the wire with help of the tensor product (Hadamard on $|\psi_0\rangle$ and Identity on $|\psi_1\rangle$). You can then merge both gates with a matrix multiplication. Note that the multiplication needs to be done in reverse order compared to the circuit (can you tell why?).
- (b) Apply the circuit to the states $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. Which famous states do we get as results?
- (c) Quantum computing is reversible, as long as we do not perform a measurement. Confirm that above circuit can be reversed by applying the gates in reverse order.

2 Simulation with Qiskit

Qiskit is an SDK developed mainly by IBM, but made available under an open-source license to allow contributions from other developers. It consists of different Python libraries for simulating and visualizing gate-based quantum algorithms, but also for execution of these algorithms remotely on IBM devices.

A detailed instruction for installation and usage can be found in the [installation guide](#).

2.1 First Steps

Once we have installed Qiskit, we can open a Jupyter notebook with Python 3 kernel (or any Python IDE like Idle or PyCharm) and check if the installation was successful. The following code should return the version of Qiskit. If you get an error instead, something is wrong with your installation.

```
1 import qiskit
2 qiskit.__version__
```

Since there are numerous tutorials for Qiskit available, we will not do a step by step introduction in the exercise sheet. Instead we recommend this [video from IBM](#). It covers the basics to create and visualize a quantum circuit and run the simulation.

If you prefer working with graphical tools, you may want to take a look at the [IBM Quantum Composer](#). It is a tool to create circuits by dragging and dropping different gates. You can run the circuit directly in the composer, or copy the automatically created Qiskit code.

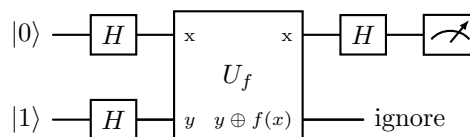
One thing that can be confusing about Qiskit at first is that the qubits are interpreted in reverse order compared to most textbooks. This is a documented feature in Qiskit with the effect that multiple qubit gates are defined differently. The CNOT gate for example is defined with the second qubit being the control bit, which leads to a different matrix representation. It is worth the time to take a few minutes and read the "Bit-ordering in the Qiskit SDK" section of [the documentation](#).

Exercise 5:

- (a) Create and visualize the circuit from Exercise 4.
- (b) Run the simulator and visualize the results in a histogram.

2.2 Deutsch Algorithm

In the lecture we have learned about the Deutsch algorithm, which is able to tell if a unary boolean function (we also called it *oracle*) is balanced or constant with only one call of the function.



Exercise 6:

- (a) Write a function in Qiskit, which is able to return a circuit representing a balanced and a constant oracle depending on a function parameter. (*Hint*: Check Lecture 5 Slide 10)
- (b) Implement the remaining algorithm and print the circuit representation.
- (c) Simulate the circuit for the balanced and constant case. How do we have to interpret the results?

2.3 Deutsch-Josza Algorithm

We can generalize the Deutsch algorithm with unary function to the Deutsch-Josza algorithm with n -ary function. Still we only need one call of this function to tell if it is balanced or constant.

Exercise 7:

- (a) Extend your function from the previous exercise to make it produce n -ary balanced or constant oracles. The easiest way is to explicitly create one balanced and one constant function. A much more complicated solution, but also a nicer one is to take an arbitrary boolean function (e. g. in form of a truth table) as input and create the belonging oracle.
- (b) Implement the remaining algorithm depending on n and print the circuit representation for $n = 4$.
- (c) Simulate the circuit for $n = 4$ and the balanced and constant case. How do we have to interpret the results?
- (d) For $n > 1$ we can construct a function which is neither constant nor balanced. What happens if we use such an oracle?

2.4 Enough about Simulation - Can We Run It on a Real Device?

Different quantum computing devices of IBM are available as a cloud service. We can access these devices for free, however it is necessary to create an account at the [IBM Quantum](#) website. If we run our code locally, we need to store our API token once with the following code (for further information look [here](#)). The token can be found on the profile page, once we have created an account. After storing the token on our local machine, we can load it for every session without the need to copy it again.

```
1 from qiskit_ibm_runtime import QiskitRuntimeService
2
3 QiskitRuntimeService.save_account(
4     token="<your-api-key>", # Use the 44-character API KEY you created and
5                             # saved from the IBM Quantum Platform Home dashboard
6     name="<account-name>", # Optional
7     instance="<IBM Cloud CRN or instance name>", # Optional
8     set_as_default=True, # Optional
9     overwrite=True, # Optional
10 )
```

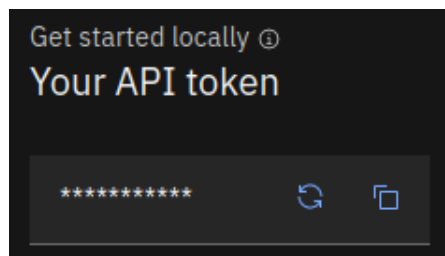


Figure 1: API token on the IBM website.

On the [IBM website](#) we can get an overview of the available devices. They are very heterogeneous and currently range from 120–156 qubits (region us-east!). By clicking on one of the devices, we can see different properties. Among others there is a connection graph. Not

all qubits are connected to each other. A connection means, that two qubits can be entangled. The connectivity is a restriction for the algorithms that can be run on the systems. We also see different error rates for the qubits. The processors are calibrated regularly and the errors that occur when gates are applied are measured.

We can run our code on the real device in the same manner that we do it on the simulator. At the end of the [video you already watched](#), there is an example about how this is done.

Exercise 8:

- Take a look on the different devices, which are available and consider which of them would be suitable to run the Deutsch-Josza algorithm for $n = 4$.
- Run the Deutsch-Josza algorithm from Exercise 7 for the balanced and constant case and $n = 4$ on one of the IBM devices and plot the results in a histogram. Note that it may take some time until your algorithm gets executed if the device is busy.
- Interpret the results, what is the difference to the results of the simulation.

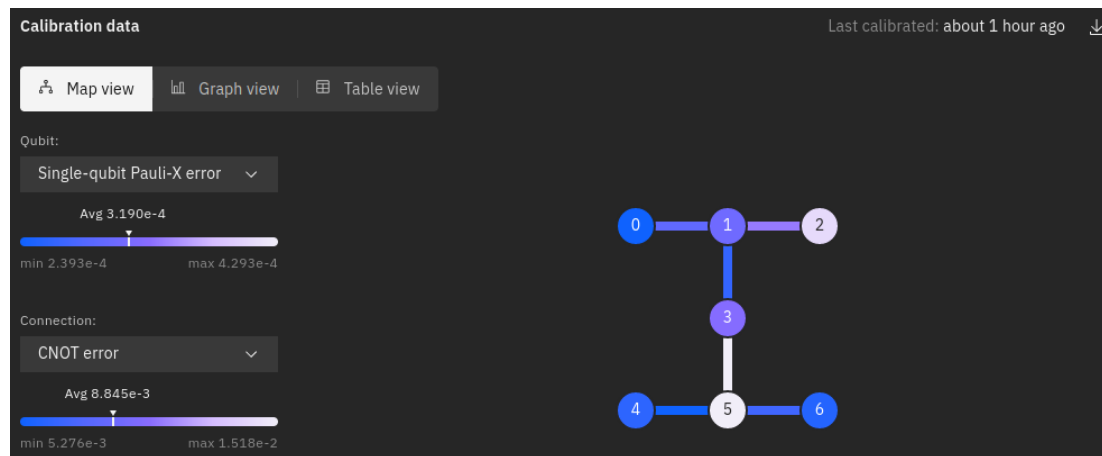


Figure 2: Topology and different properties of a former IBM processor.